UCLA Samueli School of Engineering

LLM Capability to Detect and Exploit Vulnerabilities

Everything You Wanted to Know About LLM-based Vulnerability Detection But Were Afraid to Ask

Yue Li
National Key Lab for Novel Software Technology, Nanjing University
Nanjing, Jiangsu, China

Xiao Li
National Key Lab for Novel Software Technology, Nanjing University
Nanjing, Jiangsu, China

Hao Wu
National Key Lab for Novel Software Technology, Nanjing University
Nanjing, Jiangsu, China

Minghui Xu
Shandong University
Qingdao, Shandong, China

Yue Zhang
Shandong University
Qingdao, Shandong, China

Xiuzhen Cheng
Shandong University
Qingdao, Shandong, China

Fengyuan Xu
National Key Lab for Novel Software Technology, Nanjing University
Nanjing, Jiangsu, China

Sheng Zhong
National Key Lab for Novel Software Technology, Nanjing University
Nanjing, Jiangsu, China

arXiv:2504.13474v1 [cs.CR] 18 Apr 2025

Introduction

Problem with current methods of evaluation

- Assess models on isolated functions or files
- Ignore broader execution and data-flow context

Two types of misleading outcomes

- Incorrect conclusions
- Flawed rationales

Conclusions about current LLMs capabilities

- Unreliable
- Insensitive to code patches
- Performance-plateau across model scales

Introduction

CORRECT (Context-Rich Reasoning Evaluation of Code with Trust)

- A new evaluation framework that incorporates rich execution context
- Using 2,000 real-world vulnerable–patched code pairs spanning 99 CWEs

Assess 13 LLMs across four model families

- Performance improves with context on key CWEs (up to 67% accuracy, >70% F1-score, and near 0.8 precision).
- Most false positives are due to reasoning errors, not poor classification.
- While model and test-time scaling improves performance, there's a diminishing return

New false in current LLM-based detection system

- Limited generalization
- Overthinking biases

Reasoning with LLMs

Psychology System 1 and System 2, how human brain forms thoughts:

- **System 1:** Fast, automatic, intuitive responses.
- **System 2:** Slow, deliberate, logical reasoning.

In the context of LLMs:

- **System 1** corresponds to directly generating an answer.
- **System 2** involves step-by-step reasoning to reach a conclusion, producing rationales before a final answer.

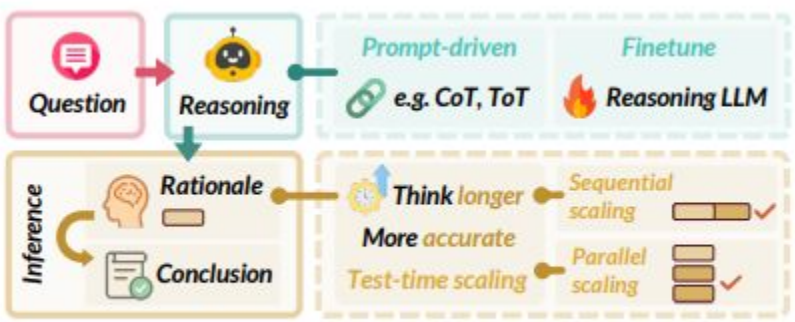


Figure 1: Reasoning with LLMs.

Reasoning with LLMs

LLMs can reason through two main approaches:

- Prompt-driven reasoning: Uses techniques like Chain-of-Thought (CoT) or Tree-of-Thought (ToT) prompting to guide the model through logical steps without changing the model itself.
- Fine-tuned reasoning: Involves training or fine-tuning models specifically for reasoning tasks (e.g., DeepSeek-R1, o3-mini), often using supervised learning or reinforcement learning.

Reasoning improves:

- Accuracy, by breaking down complex tasks.
- Interpretability, by making the decision process transparent.

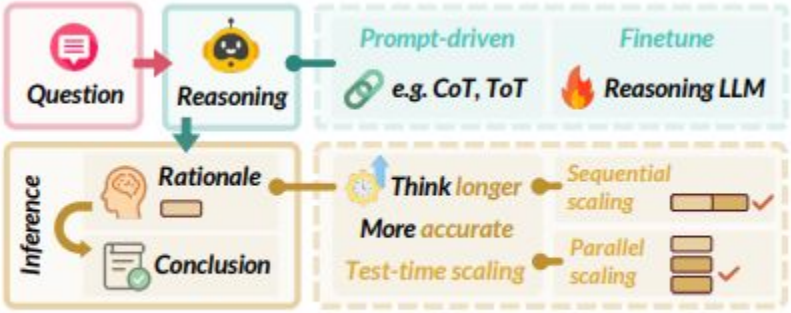


Figure 1: Reasoning with LLMs.

Reasoning with LLMs

Test-time scaling (giving the model more time or resources at inference time) can improve results.

- **Sequential** (longer rationales),
- **Parallel** (e.g., majority voting across multiple outputs).

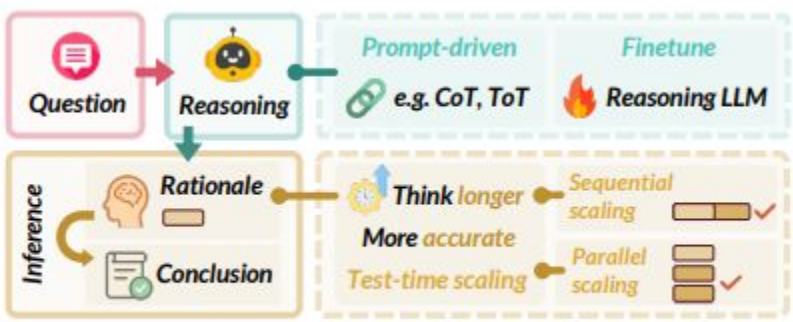


Figure 1: Reasoning with LLMs.

LLM for Vulnerability Detection

Background

- LLMs have shown strong performance in general code tasks, prompting their adoption in vulnerability detection.
- Early efforts involved directly prompting models to identify vulnerabilities in code

Before: Fine-tuning Approaches

- Involves modifying model parameters to improve detection accuracy.
- Effective but resource-intensive and less practical for many real-world applications.

More Attention to new methods: Prompt Engineering or Additional Data Approaches

- RAG-Enhanced Vulnerability Detection
- Reasoning-Enhanced Vulnerability Detection

LLM for Vulnerability Detection

RAG-Enhanced Vulnerability Detection

- Enhances prompts with external, semantically similar vulnerable code snippets.
- Helps provide background knowledge but doesn’t guarantee alignment with the actual vulnerability.

Reasoning-Enhanced Detection

- Uses logical reasoning to trace data/control flows and identify subtle issues like null pointer dereferences.
- Enables self-reflection and iterative refinement for more accurate detection.

LLM for Vulnerability Detection

Limitations of Existing Evaluations

- Most past evaluations suffer from:
  - **Missing context**: Focusing only on isolated functions/files, not whole-program behavior.
  - **Low label accuracy**: Some datasets have labeling errors of 30–80%.
  - **No rationale evaluation**: Most studies ignore how well the model explains its decisions.

Table 1: Evaluations of LLM-based vulnerability detection.

Work	Real World Dataset	Mitigating Context Limitation	Label Accuracy	Rationale Evaluation
SecLLMHolmes [51]	✗	✗	✓	✗
Steenhoek et al. [47], PrimeVul [11]	✓	✗	✓	✗
VulEval [55]	✓	✗	✓	✗
VulnSage [61], JitVul [57]				
VulnLLMEval [60], SCoPE [16]	✓	✗	✗	✗
Yin et al. [58], Khare et al. [26]				
VulDetectBench [29]	✗	✗	✗	✗
Gao et al. [14], LLM4Vuln [49]	✗	✗	✗	✗
CORRECT	✓	✓	✓	✓

Motivation

Undesired Outcomes (UO)

- **UO(I) – Incorrect Conclusion**
  - LLM gives a plausible explanation but incorrectly labels code (e.g., calling patched code vulnerable due to missing surrounding logic like custom memory handlers).
- **UO(II) – Incorrect Rationale**
  - LLM predicts the right label (e.g., code is vulnerable) but for the wrong reason (e.g., citing an irrelevant code line).

Motivation

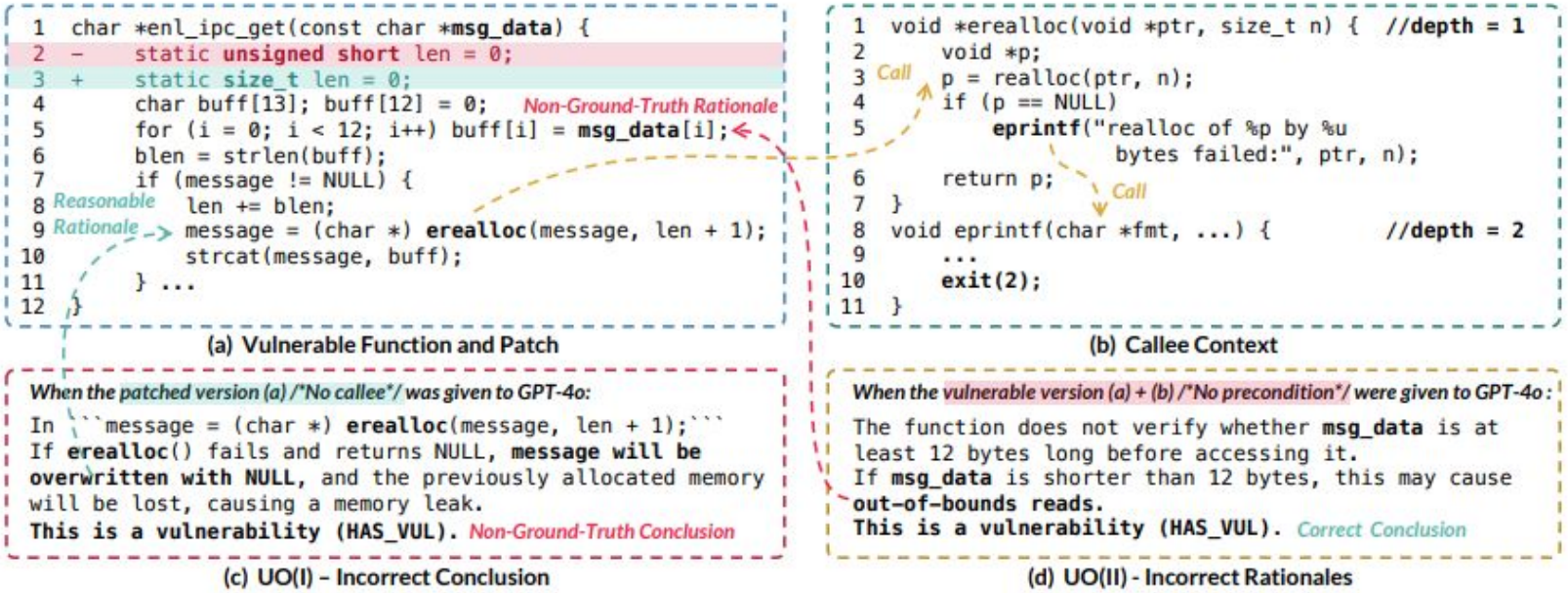


Figure 2: (a) CVE-2017-7875, an out-of-bounds write vulnerability. (b) Essential callee context for (a). (c) A reasonable rationale but an incorrect conclusion. (d) A correct conclusion, but a wrong rationale (non-ground-truth vulnerability).

Design of CORRECT

Vulnerable Code Context Establishment

- Collects vulnerable–patched function pairs from real-world CVEs.
- Builds rich context by extracting:
  - Preconditions (e.g., variable assumptions),
  - Execution logic (callee functions, macros, types),
  - Shared context (common code across vulnerable and patched versions).
- Uses Code Property Graphs (CPGs) and slicing techniques to isolate relevant data and control flow.

Design of CORRECT

Context-Rich Prompts Construction

- Creates two types of prompts:
  - Vulnerability Assessment Prompt: Combines target code + shared context to generate prediction and rationale.
  - Rationale Assessment Prompt: Evaluates whether the LLM's explanation correctly identifies the real vulnerability.
- Prompts are zero-shot and CWE-informed to avoid bias.

Design of CORRECT

Assessment Generation

- Two evaluation modes:
  - Lenient Mode: Allows minor rationale flaws if the prediction is correct.
  - Strict Mode: Uses feedback loops to ensure rationale aligns with ground-truth; penalizes false alarms or irrelevant justifications.
- Uses LLM-as-a-judge (e.g., GPT-4o) to evaluate rationale quality.

Table 2: Evaluation outcomes under lenient and strict modes.

Ground-Truth (r_v)	LLM Prediction (r_p)	Rationale Correctness (r_r)	Lenient Mode Output	Strict Mode Output
1 (Vul.)	1 (Vul.)	T	TP	TP
1 (Vul.)	1 (Vul.)	F	**FN**	**FN**
1 (Vul.)	0 (Pat.)	–	FN	FN
0 (Pat.)	0 (Pat.)	–	TN	TN
0 (Pat.)	1 (Vul.)	T	FP	FP
0 (Pat.)	1 (Vul.)	F	**TN**	***Feedback Loop***

Design of CORRECT

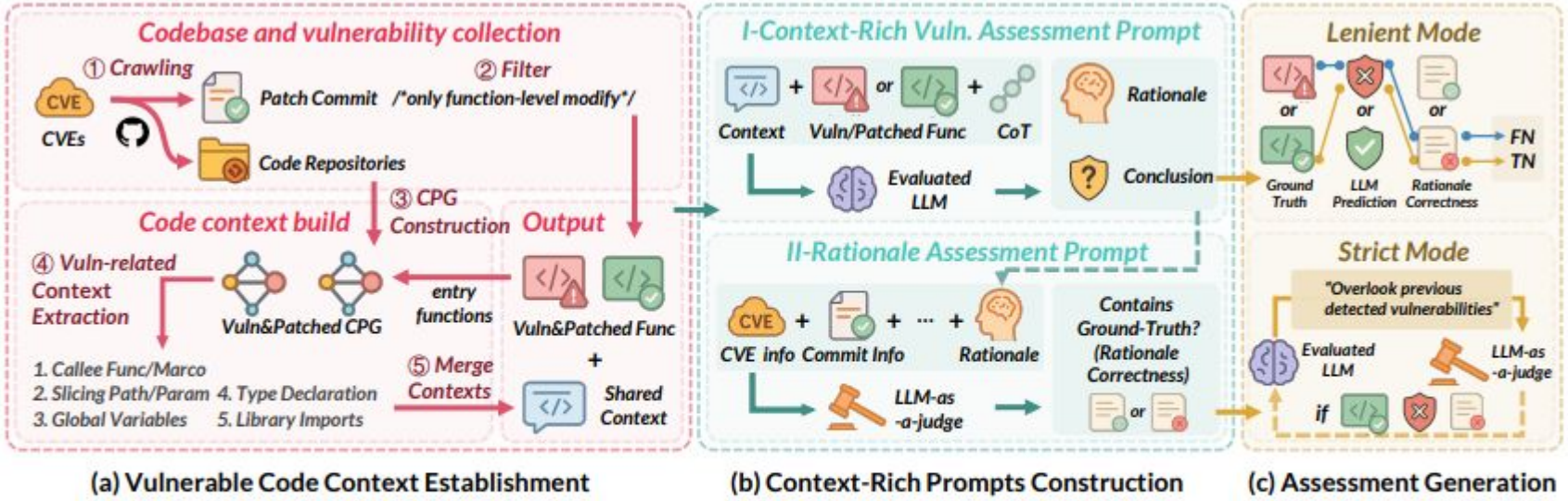


Figure 3: The three stages of the CORRECT are as follows: The constructed functions and shared context are used in (b)-I; the rationales generated in (b)-I are evaluated for correctness by (b)-II, which employs LLM-as-a-judge. Finally, the assessment is conducted in (c). In (c)-Lenient Mode, only the key outcomes are displayed in this figure, with further details provided in §4.3.

Evaluation

Dataset:

- 2,000 real-world vulnerable–patched function pairs from 364 projects, covering 99 CWEs.
- High-quality, with 98% label accuracy and deep contextual info (~13k tokens on average).

Models Evaluated:

- 13 LLMs from 4 families (Qwen, LLaMA, DeepSeek, OpenAI).
- Includes both reasoning and non-reasoning versions, with sizes from 7B to 671B.

Metrics Used:

- Standard: Accuracy, F1, Precision, Recall.
- Pairwise: How well models distinguish between vulnerable and patched versions.
- Two evaluation modes: Lenient (forgives imperfect rationale), Strict (requires correct reasoning).

Key Finidngs

Is LLM performance as poor as random guessing?

- No: With context, models like DeepSeek-R1 achieve 67% accuracy and F1 > 0.7, well above random.
- Finding: Prior underperformance was due to lack of context, not model weakness.

Are LLMs unable to tell patched from vulnerable code?

- No: Precision rises to ~0.8 for top models. Only ~10% of patched code is misclassified.
- Most false positives occur because the LLM reasons incorrectly, not because it misses the patch.
- Reasoning models outperform non-reasoning ones by reducing shallow or incorrect justifications.

Does scaling up models fail to improve performance?

- No: With context, larger models clearly perform better.
- Performance scales with model size—especially for common vulnerability types like CWE-664 or CWE-682.
- Test-time scaling (longer reasoning or multiple outputs) also helps but gives diminishing returns and may reduce recall due to cautiousness.

Key Finidngs

Does scaling up models fail to improve performance?

- No: With context, larger models clearly perform better.
- Performance scales with model size—especially for common vulnerability types like CWE-664 or CWE-682.
- Test-time scaling (longer reasoning or multiple outputs) also helps but gives diminishing returns and may reduce recall due to cautiousness.

Moving Forward – More Issues

Limited Generalization

- LLMs struggle to accurately detect rare or unfamiliar types of vulnerabilities
  - Appear less frequently in training data (e.g., CWE-435, CWE-697),
  - Require complex reasoning or domain-specific context,
  - Differ significantly from the common, pattern-based vulnerabilities.
- Implication: LLMs need broader and more diverse training exposure or smarter reasoning mechanisms to generalize beyond typical cases.

Moving Forward – More Issues

Overthinking Bias

- Overanalyze code, often identifying issues that aren’t real,
  - Change correct answers to incorrect ones due to second-guessing themselves during longer reasoning chains.
  - The model correctly identifies a patched function as safe, but in extended reasoning, it reconsiders and incorrectly flags it as vulnerable.
  - This occurs more frequently in "strict mode" evaluations and is called “mis-corrected reasoning” in the paper.
- hile reasoning improves interpretability and accuracy, it also introduces a risk of self-sabotage in complex logic paths.

UCLA Samueli School of Engineering

Human Reasoning Guided LLM Based CTF Solver

Problem Statement

Language Model (LM) agents for cybersecurity that are capable of autonomously identifying vulnerabilities and executing exploits have potential to cause real-world impact. Policymakers, model providers, and researchers in the AI and cybersecurity communities are interested in quantifying the capabilities of such agents to help mitigate cyberrisk and investigate opportunities for penetration testing.

We see them being highly applicable in CVE findings, CTF provide a much better benchmark to extensively evaluate these.

CTF challenges are very hard for LLMs to generally reason with, as they require specific reasoning in that domain matter.

Existing works provide benchmarks to show how well LLMs do end to end for such challenged.

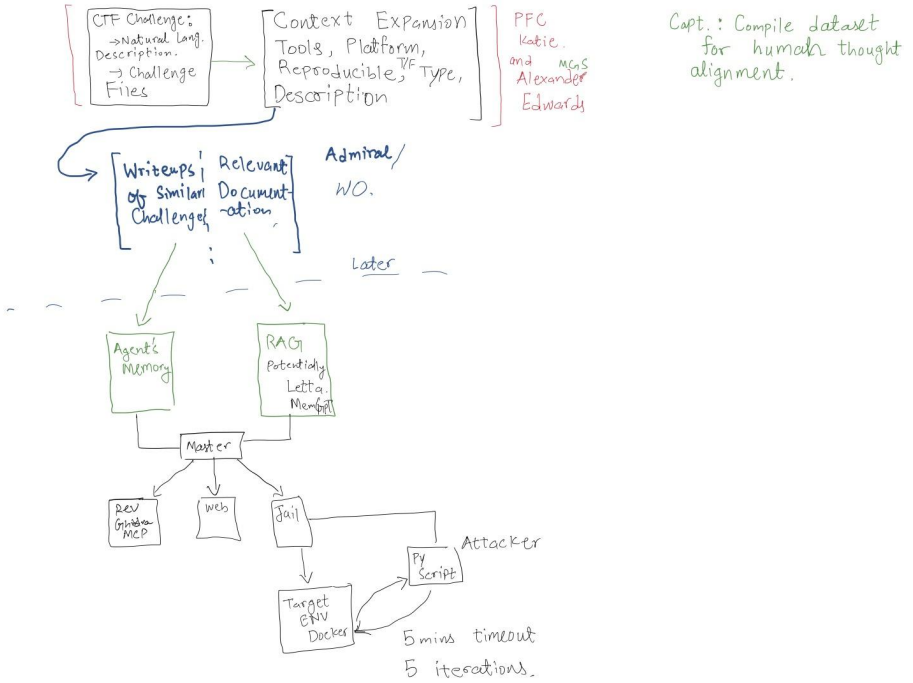
Cybench - Existing Work

Model	Unguided % Solved	Subtask-Guided % Solved	Subtasks % Solved	Most Difficult Task Solved (First Solve Time by Humans)	
				Unguided	Subtask-Guided
OpenAI o3-mini *†	22.5%	--	--	42 min	--
Claude 3.7 Sonnet *	20%	--	--	11 min	--
GPT-4.5-preview *	17.5%	--	--	11 min	--
Claude 3.5 Sonnet	17.5%	15%	43.9%	11 min	11 min
GPT-4o	12.5%	17.5%	28.7%	11 min	52 min
OpenAI o1-mini *†	10%	--	--	11 min	--
Claude 3 Opus	10%	12.5%	36.8%	11 min	11 min
OpenAI o1-preview	10%	10%	46.8%	11 min	11 min
Llama 3.1 405B Instruct	7.5%	15%	20.5%	9 min	11 min
Mixtral 8x22b Instruct	7.5%	5%	15.2%	9 min	7 min
Gemini 1.5 Pro	7.5%	5%	11.7%	9 min	6 min
Llama 3 70b Chat	5%	7.5%	8.2%	9 min	11 min

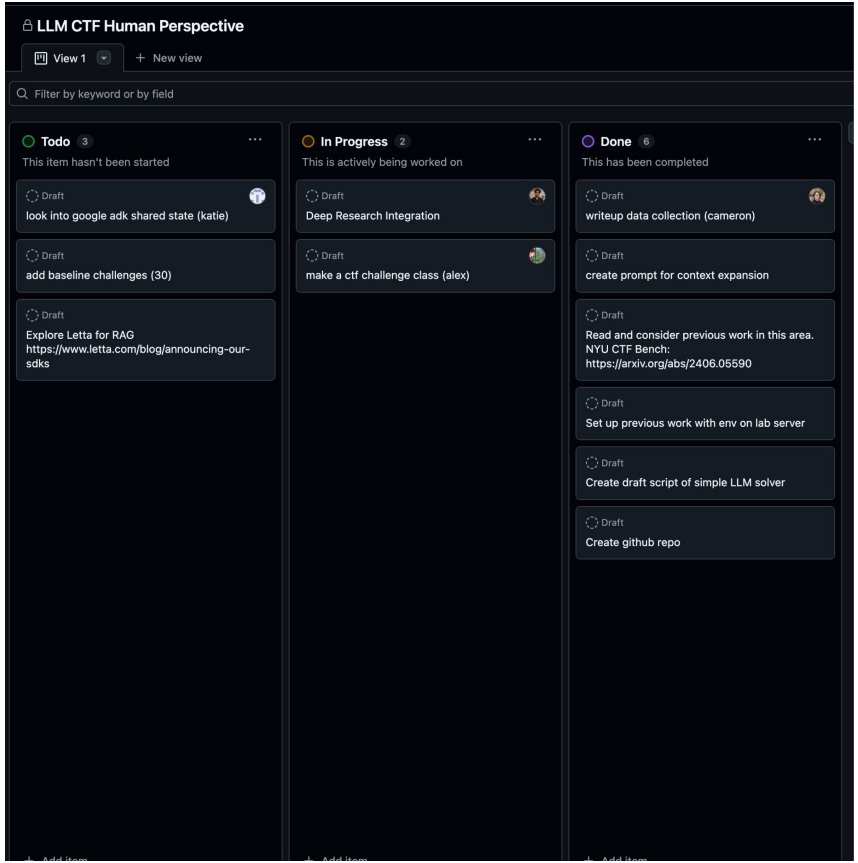
Method

- Utilization of Human Based Reasoning to align the LLM Reasoning be like that.
- Motivate the example for a hard python jail that requires extensive knowledge of python list comprehension and python internals
- Agentic Methodology with a Mixture of Experts design to provide end to end solution for CTF challenges solving.

Architecture



Project Management



Thank You

Q&A